Sliding Window

07 February 2023 21:11

N = 4, K = 2

Arr = [100 , 200, 300 , 400]

{Subarray always be continuous}

k=2, 100,200 = 300
200,300 = 500
300,400 = 700 } 700

100, 200, 300, 400

i ---- j

j-i+1

k

Brute force

O(n*k)

k==n

O(n*n)

k<n

O(n*k)

k

j-i+1

0-0+1=1

k=3

1-0+1=2=k ✗

2-0+1=3=k ✓

(j-i+1 != k)

j++

(j-i+1 == k)

{600}

100, 200, 300, 400

k=2

subarray = 100, 200, 300, 400,

Subsequence 100, 200, 300, 400, 200, 300, 400, 100, 200, 300, {100, 200, 300}

ans = MIN

sum=0

sum=100

sum=300

400

600

sum=300

ans=-∞

sum > ans

ans=sum

sum -= arr[i]

i++;

j++;

(j-i+1 == k)

ans = MAX(ans, sum)

(j-i+1 != k)

j++

0-0+1 != 2

j++

1-0+1=2

ans=-∞

s > ans

ans = s

First negative integer in every window of size k

arr[i] > 0

cont

{12 , -1 , -7 , 8 , -15 , 30 , 16 , 28}

[K = 3]

deq:

-1 | -7 | -15

k

0

ans:

-1 | -1 | -7 | -15 | -15 | 0

arr

template

1. i=0, j=0
2. while(j<n)
3. j-i+1 != k
   j++
4. j-i+1 == k
   ans[i++] = arr[j]
5. return ans.

-8 , 2 , 3 , -6 , 10

k=2

[-8, 0, -6, -6]

deq:

-8 | -6

ans:

-8 | 0 | -6 | -6

Distinct Numbers in Window

Example Explanation

Explanation 1:

```
A=[1, 2, 1, 3, 4, 3] and B = 3
All windows of size B are
[1, 2, 1]
[2, 1, 3]
[1, 3, 4]
[3, 4, 3]
So, we return an array [2, 3, 3, 2].
```

Explanation 2:

```
Window size is 1, so the output array is
[1, 1, 1, 1].
```

k = 3

1 2 1 3 4 3

i j

⇒ 4 1 2 3 k = 1

[1, 1, 1, 1]

3 - 1
4 - 1

[2]
[3]
[3]
[2]

size = 2

j - i + 1 == k

4 = 4

Permutation in String

codechef (i, j) edhc (s1)

① s1 length > s2 length. false

c → 0
h → 0
d → 0
e → 0

Screen clipping taken: 08-02-2023 21:55

mp.size == 0
↳ True

```
while(j < n)
{
    //ca
    if(mp.find(s2[j]) != mp.end()) // means we found
    {
        mp[s2[j]] -- ;
        if(mp[s2[j]] == 0)
        {
            size -- ;
        }
    }
    if(j - i  + 1 != k )
    {
        j ++ ;
    }
    else if(j - i + 1 == k)
    {
        if(size == 0) return true;

        if(mp.find(s2[i]) != mp.end())
        {
            mp[s2[i]] ++ ;
            if(mp[s2[i]] == 1) size ++;
        }
        i++;
        j++;
    }
}
return false;
```

code (i, j) od

2 → 1 → 0
d → 0
o → 0
2

s2 = forxxorfxdofr (i, j)

s1 = for

f = 1 → 0
o = 1 → 0
r = 1 → 0

Minimum Window Substring

ADOBECODEBANC (i, j)

ABC

ADOBEC , 6 ± 2

a → 1 → 0 k = 3 → 2 → 1 → 0
b → 1 → 0
c → 1 → 0

ADOBEC 6

⇒ BANC 4

3 → 2 → 1 → 0 → 1 → 0

Smallest distinct window

AABBBCBBAC (i, j)

ABC.

a → 1 → 0 → 1 → 2
b → 1 → 2 → 3 → 2 → 0
c → 1 → 0

AABBBC → 6

Sliding Window Max

K = 3

104

arr:

O(n)

O(1)

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position                Max
---------------               -----
[1  3  -1] -3  5  3  6  7       3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
```

a[i] == dq.front

a[j] > dq[back]

1 3 -1 -3 5 3 6 7

i

j

3, 3, 5, 5, 6, 7

Screen clipping taken: 10-02-2023 22:22

arr:

Max

O(n)

O(1) push-back

O(n) push-front

O(1) pop-back

C++

list, dq

dq:

O(1)

O(1)

O(1)

O(1)

```
//define ds
deque<int> dq ;
vector<int> ans;
while(j < n)
{
    //cal
    if(dq.size() == 0){
        dq.push_back(nums[j]);
    }
    else{
        while(dq.size() > 0 &&  dq.back() < nums[j])
        {
            dq.pop_back() ;
        }
        dq.push_back(nums[j]) ;
    }
    if(j - i  + 1 != k){
        j ++ ;
    }
    if(j - i + 1 == k){
        ans.push_back(dq.front()) ;
        if(nums[i] == dq.front()){
            dq.pop_front() ;
        }
        i ++ ;
        j ++ ;
    }
}
return ans ;
```

[9, 11]

k = 2

i

j

9 < 11

11 > 9

11

Longest Sub-Array with Sum K

i

k

sum = 15

Longest Sub-Array with Sum K

10, 5, 2, 7, 1, 9

i

j

K = 15

k

sum = 15

sum = 0 to 15

5 2 7 1

ans = max(ans, j-i+1).

(sum == k)

(sum < k)

(sum > k)

15 5

{ Count of subarray }

N = 3, K = 2

Arr[] = {3, 2, 1}

[3] [3 2] [3 2 1]

[2] [2 1] [1]

N = 4, K = 1

Arr[] = {1, 2, 3, 4}

duplicate count

[3 3 1]

3, 33, 331

3, 31, 1

N = 3 → subarray = 6

3 2 1

3 + 2 + 1 ⇒ 1+2+3+ . . . n

⇒ n(n+1)/2 = 3 × (3+1)/2 = 6

N = 3 k = 2

arr: {3, 2, 1}.

[3] [3 2] [3 2 1]

[2] [2 1] [1]

Given an array of **N** positive integers **Arr$_1$, Arr$_2$ ..... Arr$_n$**. The value of each contiguous subarray of given array is the **maximum element present in that subarray**. The task is to return the number of subarrays having value strictly greater than **K**.

k = 2

3 > 2 ✓ ⇒ [6, 1, 2]

{val = 6}

= val > K

[3, 2] ⇒ 3 > 2 ✓

[3, 2, 1] ⇒ 3 > 2 ✓

[2] > 3 ✗ K

[2 1] = 2 > ~~3~~ K ✗

1 = 1 > k ✗

a(i) <= k way

c+1

[4 3]

[4] [4 3] [3]

⇒ 3

⇒ 2 * (2+1)/2

x+3/x = 3

k = 4

c = 0

[4 3] 5, 6, [1, 2, 3]

ans = 1

✗ 2

c*(c+1)/2

1*(1+1)/2 = 1

c*(c+1)/2

n*(n+1)/2 -

> k ✓

<= k → uelen

total - <= k

n*(n+1)/2 — —

ans = 15

uelen = 0

total sub

13*(13+1)/2

13*14/2

91 - 25 = 66 Ans

k = 5

6, 7, [3, 2, 1, 4, 5], 9, 8, 1, 2, 3, 4,

c = 5

5*(5+1)/2 = 5*6/2 = 15 +

4*(4+1)/2

24*5/2

210 = 25